

Minimum Number of coins to get the Sum

T.C : Exp

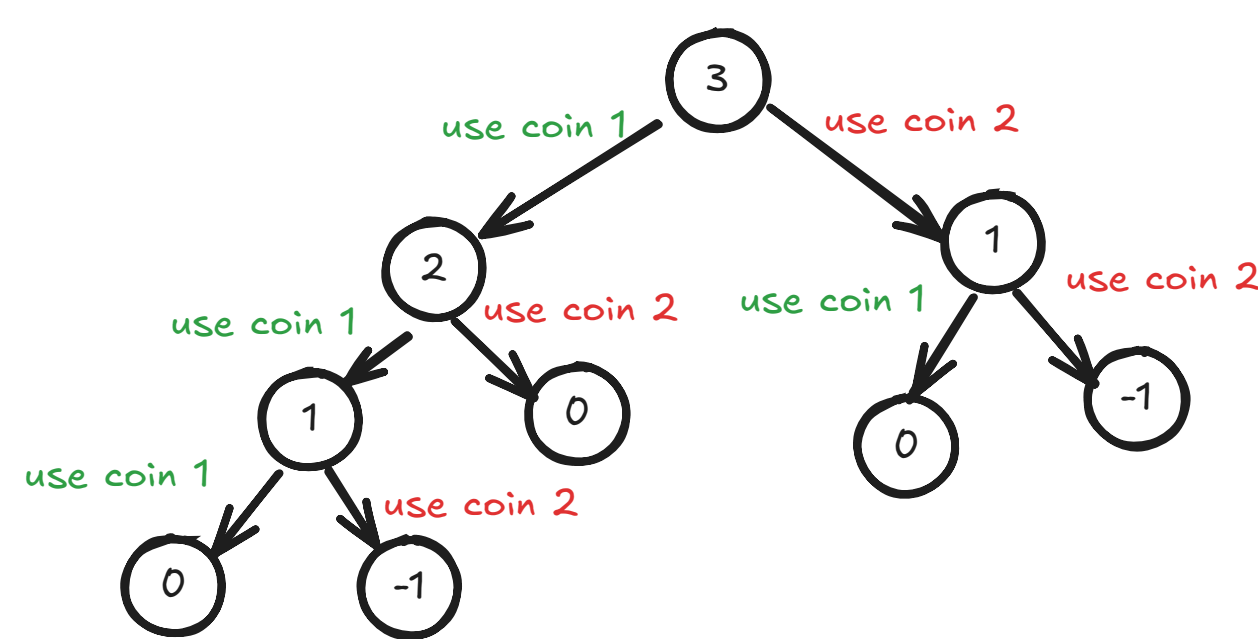
- An Array is given which contains integers
- Sum is also given , and we need to make Sum using these integers
- We can use a particular integer any number of times
- We need to find the minimum coins needed to make the sum.

Base Case :

- If you have got 0, means that you have completed the sum, and that whole chain is actually one of the way.

- If you have got value less than 0 or negative value, it means that this chain is not creating the sum. and this is not one of the way

Example : nums = [1,2], Sum = 3



```
int solveRec( vector<int> &nums, int x ){

    // base case
    if( x == 0 ) {
        return 0
    }

    if( x < 0 ){
        return INT_MAX;
    }

    // Recursive Relation
    int mini = INT_MAX;
    for( int i = 0; i < nums.size(); i++ ){
        int ans = solveRec( nums, sum - nums[i] );
        if( ans != INT_MAX){
            mini = min( mini, ans + 1);
        }
    }

    return mini;

int minCount( vector<int> &nums, int x){
    int ans = solveRec(nums,x);
    if( ans == INT_MAX){
        return -1;
    }
    else{
        return ans;
    }
}
```

SIMPLE RECURSION LOGIC

- I need to think of a way such that, as you can see when the 0 is hit, it should return 1 to upper node.

- Look at the flow, 3 -> 2 -> 1 -> 0
0 should return 1 to node 1, 1 should return 2 to node 2, 2 should return 3 to node 3 , 3 Ways to achieve the sum .

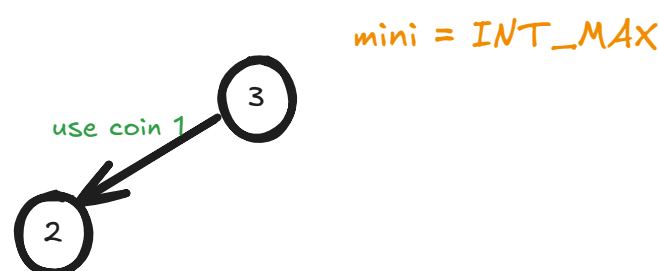
- Look at the flow 3 -> 1 -> 0
0 should return 1 to the node 1, 1 should return 2 to the node 3, and hence there are 2 ways to achieve the sum.

- Look at the flow, 3 -> 2 -> 0, 2 ways,

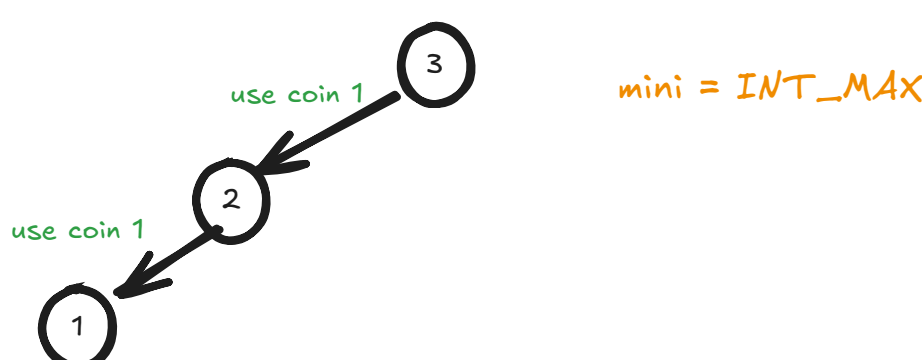
- Answer should be 2 as minimum way to achieve the sum.

- from the main function the call was for solve([1,2] , 3) or solve(nums,3)

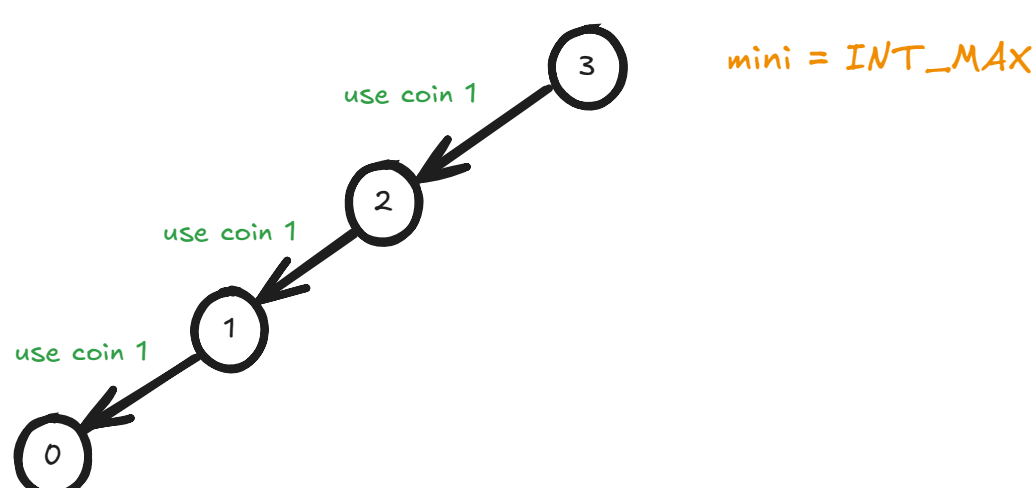
- Coin 1 is taken and then the call is given for solve(nums, 2).



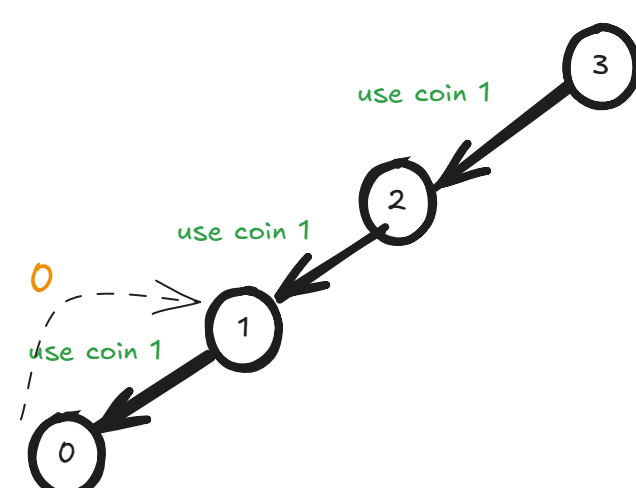
- at the sum 2, coin 1 is chosen and the call is given for solve(nums,1)



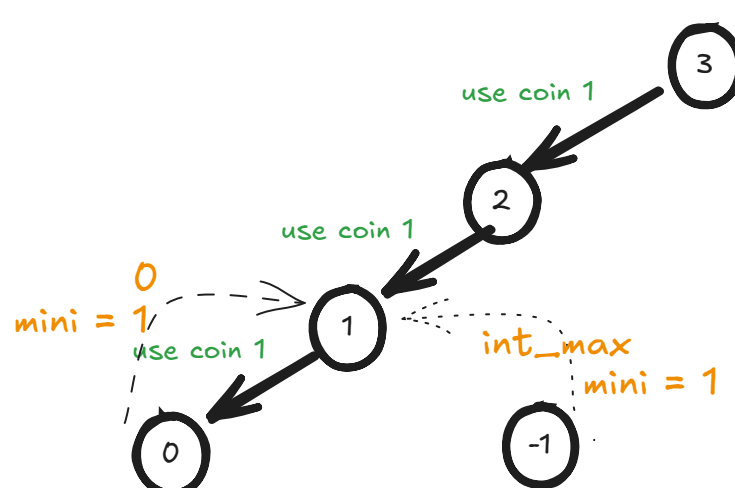
- at the sum 1, coin 1 is chosen and the call is for solve(nums, 0).



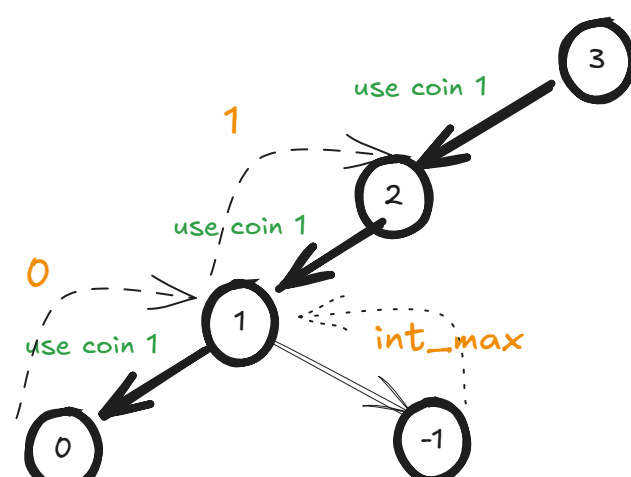
- When the solve(nums, 0) is called, this returns 0 as ans.



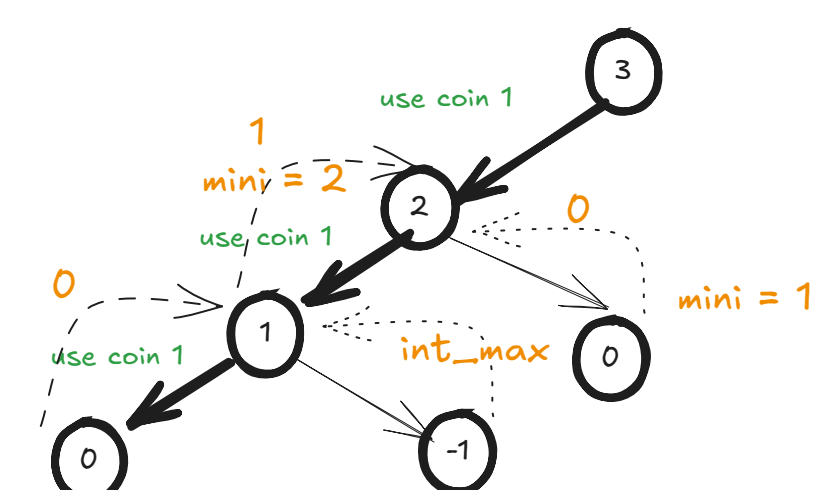
- solve(nums, 1) had called it, so now you can see mini is min(INT_MAX, 0 + 1) mini becomes 1, Now as it was inside the loop , hence now coin 2 is there and now call for solve(nums, -1) is given, which returns INT_MAX and hence mini is still 1.



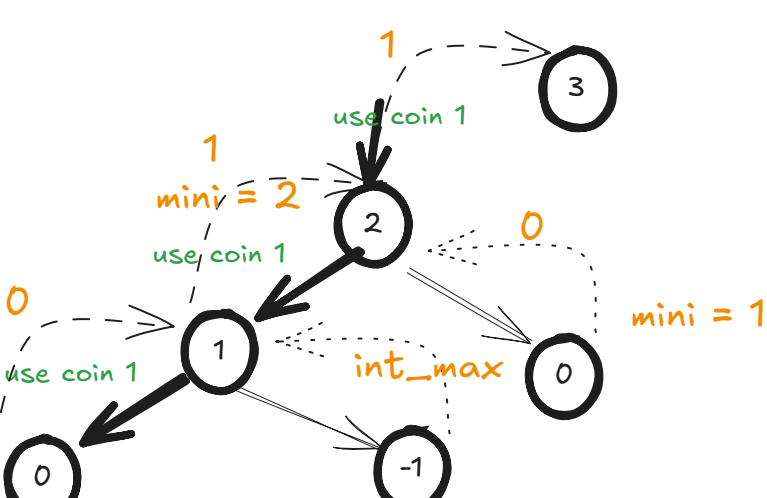
- Now the for loop is ended and Node 1, returns mini which was 1, to the Node 2



- solve(nums, 2) had called solve(nums,1) and it has returned 1 to it, Now as it is not int_max , mini becomes 2, Now it is on the for loop so now it encounters coin2, and now calls solve(nums, 0), which returns 0, and it is not int_max ans is 0, so mini becomes min(2, 0 + 1) which is 1. so 2 returns 1 to the node 3.



- So Now, solve(nums, 2) returns 1 to the node 3



- Solve(nums, 3) : It had, min as INT_ MAX and then called solve(num,2) which returned 1, now mini becomes min(int_max,1 + 1) hence becomes 2. Now it is inside the for loop and now it will call solve(nums, 1), We have solved this part earlier that it will return 1, (in the code, this tree forms again), So now min was 2, we called solve(nums, 1) which will return 1, and now the min(2,2) is 2.

- Solve(nums,3) will return 2. ^{Ans}

Dynamic Programming : Adding Memoization to this code (TOP DOWN APPROACH)

T.C. = n*x

S.C. = x

```
#include <bits/stdc++.h>

int solveTab(vector<int> num, int x){
    vector<int> dp(x + 1, INT_MAX); // Fill the vector with INT_MAX
    dp[0] = 0;
    for( int i = 1; i<=x; i++){
        for( int j = 0; j < num.size(); j++){
            if( i - num[j] >= 0 & dp[i - num[j]] != INT_MAX){
                dp[i] = min( dp[i], 1 + dp[i - num[j]]);
            }
        }

        if( dp[i] == INT_MAX){
            return -1;
        }

        return dp[x];
    }

    int minimumElements(vector<int> &num, int x)
    {
        return solveTab(num,x);
    }
}
```

Step 1 : Make the dp array, and pass it as argument

Step 2 : After recursive call, store the elements in the dp array

Step 3 : After the base case, return element of the dp array if it exists checking if it is -1 or not.

-- You generally make dp array of the values of the actual node.

Dynamic Programming : Tabulation (BOTTOM UP APPROACH)

T.C. = n*x

S.C. = x

```
#include <bits/stdc++.h>

int solveTab(vector<int> num, int x){
    vector<int> dp(x + 1, INT_MAX); // Fill the vector with INT_MAX
    dp[0] = 0;
    for( int i = 1; i<=x; i++){
        for( int j = 0; j < num.size(); j++){
            if( i - num[j] >= 0 & dp[i - num[j]] != INT_MAX){
                dp[i] = min( dp[i], 1 + dp[i - num[j]]);
            }
        }

        if( dp[i] == INT_MAX){
            return -1;
        }

        return dp[x];
    }

    int minimumElements(vector<int> &num, int x)
    {
        return solveTab(num,x);
    }
}
```

- In this you solve all the basic subproblems from the bottom

- you set dp[0] = 0

- you found dp[1] = 1

- for i =2 , or for any i, there will be 2 options 1 and 2.

- i = 2 ; j = 1 and 2

- for j = 1; dp[2] = min(INT_MAX, 1 + dp[2 - 1]) = 2

- for j = 2; dp[2] = min(2, 1 + dp[2 - 2]) = min(2, 1) = 1.

Here more space optimization is not possible, as we have many coins, and there is no fixed range that dp[i - num[j]] is there.